MINI PROJECT REPORT
EXPENSE TRACKER USING JAVA AND MYSQL

Name : Harshitha J
USN: 1RUA25BCA0041

1. Introduction
The Expense Tracker is a desktop application developed using Java Swing and MySQL. It helps users record, view, update, and delete their daily expenses. MySQL is used to store expense details permanently.

2. Objectives
- To develop a simple expense management application.
- To connect a Java application to a MySQL database using JDBC.
- To perform database operations such as INSERT, SELECT, UPDATE, and DELETE.
- To calculate and display the total expenses.

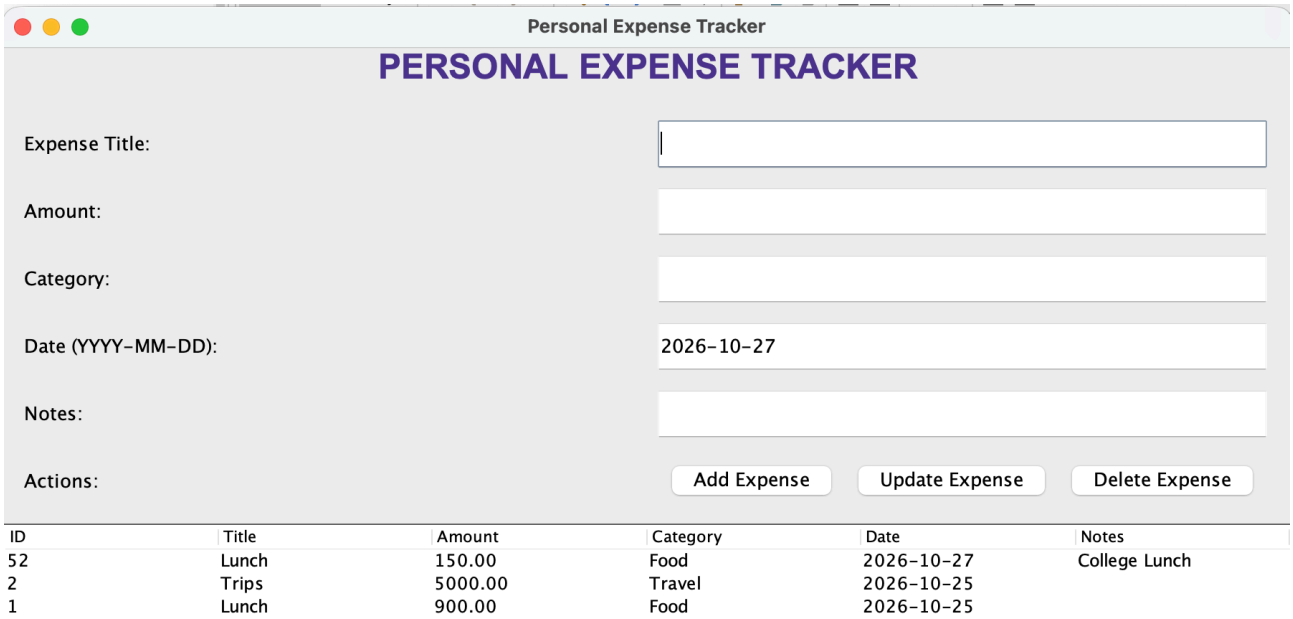
3. Technologies Used
- Programming Language: Java
- User Interface: Java Swing
- Database: MySQL
- Database Connectivity: JDBC
- IDE: Apache NetBeans

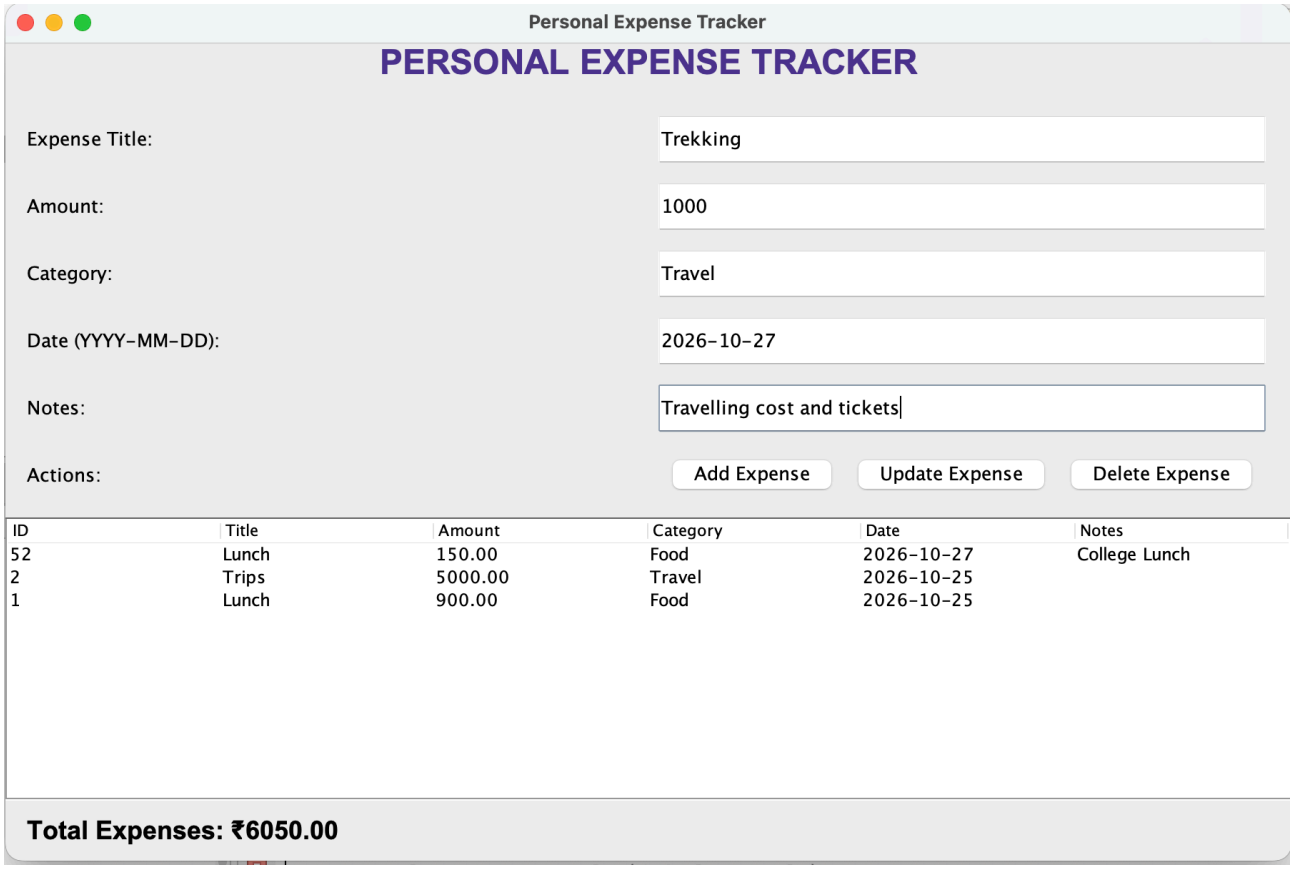
4. Database Design
Database Name: expense_tracker

Table Name: expenses

Field	Data Type
id	INT, Primary Key, Auto Increment
title	VARCHAR(100)
amount	DECIMAL(10,2)
category	VARCHAR(50)
expense_date	DATE
notes	VARCHAR(255)

5. Application Implementation
The application provides options to add new expenses, view saved expenses, update existing records, delete records, and calculate the total amount. The application connects directly to MySQL through JDBC.


Personal Expense Tracker
PERSONAL EXPENSE TRACKER
Expense Title:
Amount:
Category:
Date (YYYY-MM-DD):
2026-10-27
Notes:
Actions:
Add Expense
Update Expense
Delete Expense
ID Title Amount Category Date Notes
52 Lunch 150.00 Food 2026-10-27 College Lunch
2 Trips 5000.00 Travel 2026-10-25
1 Lunch 900.00 Food 2026-10-25
Total Expenses: ₹6050.00


Personal Expense Tracker
PERSONAL EXPENSE TRACKER
Expense Title:
Trekking
Amount:
1000
Category:
Travel
Date (YYYY-MM-DD):
2026-10-27
Notes:
Travelling cost and tickets
Actions:
Add Expense
Update Expense
Delete Expense
ID Title Amount Category Date Notes
52 Lunch 150.00 Food 2026-10-27 College Lunch
2 Trips 5000.00 Travel 2026-10-25
1 Lunch 900.00 Food 2026-10-25
Total Expenses: ₹6050.00

ID	Title	Amount	Category	Date	Notes
53	Trekking	1000.00	Travel	2026-10-27	Travelling cost and tick...
52	Lunch	150.00	Food	2026-10-27	College Lunch
2	Trips	5000.00	Travel	2026-10-25	
1	Lunch	900.00	Food	2026-10-25	

My-SQL and Apache netbeans connection

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    public static Connection getConnection() throws SQLException {
        String url = "jdbc:mysql://localhost:3306/expense_tracker";
        String user = "root";
        String password = "ExpenseTracker";

        return DriverManager.getConnection(url, user, password);
    }

    public static void main(String[] args) {
        try (Connection con = getConnection()) {
            System.out.println("MySQL Connected Successfully!");
        } catch (SQLException e) {
            System.out.println("Connection Failed!");
            e.printStackTrace();
        }
    }
}
```

```
Run (prog1) × Run (ExpenseTracker) × Run (DBConnection) ×
--- exec:3.6.3:exec (default-cli) @ prog1 ---
MySQL Connected Successfully!
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  0.421 s
Finished at: 2026-10-27T16:03:35+05:30
------------------------------------------------------------------------
```

```java
import ExpenseTracker.DBConnection;
import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class ExpenseTracker extends JFrame {

    JTextField titleField, amountField, categoryField;
    JTextField dateField, notesField;
    JTable table;
    DefaultTableModel model;
    JLabel totalLabel;
    int selectedId = -1;

    public ExpenseTracker() {
        setTitle("Personal Expense Tracker");
        setSize(900, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        JLabel heading = new JLabel("PERSONAL EXPENSE TRACKER",
                SwingConstants.CENTER);
        heading.setFont(new Font("Arial", Font.BOLD, 24));
        heading.setForeground(new Color(75, 50, 140));
        add(heading, BorderLayout.NORTH);

        JPanel form = new JPanel(new GridLayout(6, 2, 8, 8));
        form.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));

        titleField = new JTextField();
        amountField = new JTextField();
        categoryField = new JTextField();
        dateField = new JTextField(java.time.LocalDate.now().toString());
        notesField = new JTextField();
```

```
        form.add(new JLabel("Expense Title:"));
        form.add(titleField);
        form.add(new JLabel("Amount:"));
        form.add(amountField);
        form.add(new JLabel("Category:"));
        form.add(categoryField);
        form.add(new JLabel("Date (YYYY-MM-DD):"));
        form.add(dateField);
        form.add(new JLabel("Notes:"));
        form.add(notesField);

        JButton addButton = new JButton("Add Expense");
        JButton updateButton = new JButton("Update Expense");
        JButton deleteButton = new JButton("Delete Expense");
        JButton clearButton = new JButton("Clear Fields");

        JPanel buttons = new JPanel(new FlowLayout());
        buttons.add(addButton);
        buttons.add(updateButton);
        buttons.add(deleteButton);
        buttons.add(clearButton);

        form.add(new JLabel("Actions:"));
        form.add(buttons);

        String[] columns = {
            "ID", "Title", "Amount", "Category", "Date", "Notes"
        };

        model = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };
```

```
        table = new JTable(model);
        table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        JScrollPane scrollPane = new JScrollPane(table);

        totalLabel = new JLabel("Total Expenses: ₹0.00");
        totalLabel.setFont(new Font("Arial", Font.BOLD, 18));
        totalLabel.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));

        JPanel center = new JPanel(new BorderLayout());
        center.add(form, BorderLayout.NORTH);
        center.add(scrollPane, BorderLayout.CENTER);
        center.add(totalLabel, BorderLayout.SOUTH);

        add(center, BorderLayout.CENTER);

        addButton.addActionListener(e -> addExpense());
        updateButton.addActionListener(e -> updateExpense());
        deleteButton.addActionListener(e -> deleteExpense());
        clearButton.addActionListener(e -> clearFields());

        table.getSelectionModel().addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()
                    && table.getSelectedRow() != -1) {
                int row = table.getSelectedRow();
                selectedId = Integer.parseInt(
                        model.getValueAt(row, 0).toString());

                titleField.setText(model.getValueAt(row, 1).toString());
                amountField.setText(model.getValueAt(row, 2).toString());
                categoryField.setText(model.getValueAt(row, 3).toString());
                dateField.setText(model.getValueAt(row, 4).toString());

                Object notes = model.getValueAt(row, 5);
                notesField.setText(notes == null ? "" : notes.toString());
            }
```

```
        }
    });

    loadExpenses();
}

private boolean validateFields() {
    if (titleField.getText().trim().isEmpty()
            || amountField.getText().trim().isEmpty()
            || categoryField.getText().trim().isEmpty()
            || dateField.getText().trim().isEmpty()) {
        JOptionPane.showMessageDialog(this,
                "Please fill in title, amount, category and date.");
        return false;
    }

    try {
        double amount = Double.parseDouble(amountField.getText().trim());

        if (!Double.isFinite(amount) || amount <= 0) {
            JOptionPane.showMessageDialog(this,
                    "Amount must be a positive number.");
            return false;
        }

        java.time.LocalDate.parse(dateField.getText().trim());

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this,
                "Enter a valid amount and date (YYYY-MM-DD).");
        return false;
    }

    return true;
}
```

```
private void addExpense() {
    if (!validateFields()) return;

    String sql = "INSERT INTO expenses "
            + "(title, amount, category, expense_date, notes) "
            + "VALUES (?, ?, ?, ?, ?)";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, titleField.getText().trim());
        ps.setBigDecimal(2, new java.math.BigDecimal(
                amountField.getText().trim()));
        ps.setString(3, categoryField.getText().trim());
        ps.setDate(4, Date.valueOf(dateField.getText().trim()));
        ps.setString(5, notesField.getText().trim());

        ps.executeUpdate();

        JOptionPane.showMessageDialog(this, "Expense added successfully!");
        clearFields();
        loadExpenses();

    } catch (Exception ex) {
        showError(ex);
    }
}

private void loadExpenses() {
    model.setRowCount(0);
    double total = 0;

    String sql = "SELECT * FROM expenses ORDER BY expense_date DESC, id DESC";

    try (Connection con = DBConnection.getConnection();
         Statement st = con.createStatement();
```

